

# Lesson 1

## Intro to React, App Installation & Vite

Code the Dream — React

# Today's Session

## EXPLORE

- What React is and why it exists
- Declarative vs. imperative programming
- Components and the Virtual DOM
- Setting up a project with Vite
- Project structure and the dev server

## APPLY

- Start your Vite dev server
- Build an "About Me" component
- Use variables and map over an array in JSX

By the end: understand what React is for, have a running Vite app, and render your own JSX to the browser.

# What Is React?

React is a **JavaScript library** for building user interfaces.

- Created by Meta, maintained by Meta and the open source community
- Used to build **single-page applications (SPAs)** — web apps that update without full page reloads
- Takes a **declarative** approach: you describe what the UI should look like, React handles the rest

You have likely used apps built with React: Facebook, Instagram, Airbnb, and many others.

# The Problem React Solves

Building interactive UIs without a library is complicated.

## Without React (imperative)

- Manually create and remove elements
- Manually manage event listeners
- Manually update the DOM when data changes

## With React (declarative)

- Describe what the UI should look like
- React listens for changes
- React updates the DOM for you

# What React Takes Care Of

When you use React, the library handles:

- Assembling **components** to render the UI
- Listening for **user events** — clicks, typing, hover
- Updating **state** when events happen
- Automatically **re-rendering** the UI when state changes

You focus on describing the UI. React handles the updates.

# Components

A **component** is a reusable piece of UI.

- React apps are built by combining many components
- Each component handles **one part** of the page
- Components can be small (a button) or large (a whole page section)

```
function Greeting() {  
  return <h1>Hello, world!</h1>;  
}
```

Think of components like LEGO bricks — each one does one thing, and you combine them to build something bigger.

# The Virtual DOM

React uses a **Virtual DOM** to update the page efficiently.

1. When state changes, React updates a **lightweight copy** of the DOM in memory
2. React **compares** the copy to the real DOM (called "diffing")
3. React updates **only the parts that changed**

This is faster than rebuilding the entire page on every change.

# Check for Understanding

Which best describes the **declarative** approach React uses?

- A) The developer writes code to directly add, remove, and update DOM elements
- B) The developer describes what the UI should look like, and React handles the updates
- C) React automatically generates the entire UI without any developer input
- D) The developer writes instructions for every user interaction one by one



# Starting a React Project with Vite

**Vite** is the build tool and development server we use with React.

- Provides a fast local development server
- Transforms JSX into JavaScript the browser understands
- Updates the browser instantly when you save — no restart needed (**HMR**)

To create a new React project:

```
npx create-vite@latest --template react .
```

Then install dependencies:

```
npm install
```

# Starting the Dev Server

After installing, start the development server:

```
npm run dev
```

Open your browser to `http://localhost:5173` — you should see the default Vite + React starter page.

**:5173** is the port Vite uses locally. You'll use this URL every time you work on your app.

# Project Structure

```
.
├── index.html      ← entry point for the app
├── package.json    ← project info and dependencies
├── public/         ← static files (images, fonts)
├── src/
│   ├── App.jsx     ← main component
│   ├── App.css
│   ├── main.jsx    ← mounts React to the page
│   └── index.css
└── vite.config.js  ← Vite configuration
```

Most of your work will happen inside the `src/` folder.

# The Development Workflow

Keep the development server running while you work.

## The loop:

1. Edit a file in `src/`
2. Save (`Ctrl+S` / `Cmd+S`)
3. Vite detects the change — the browser updates automatically
4. Read the output, fix any errors, repeat

If the browser stops updating, check your terminal — a syntax error likely stopped the server.

# Check for Understanding

You scaffold a new Vite project and open it in VS Code. What do you need to do **before** you can run the dev server?

- A) Run `npm start`
- B) Run `npm install`, then `npm run dev`
- C) Open `index.html` directly in the browser
- D) The dev server starts automatically — no extra steps needed

# Now: Apply

Let's get a React component running in the browser.

## This Week's Exercise — "About Me" Component

You'll build a simple **About Me** page using JSX inside the exercise repo.

**File to edit:** `src/exercises/lesson-01/studentWork.jsx`

Your component should include:

- A variable for your name
- A variable for your age
- An array of your hobbies
- JSX that renders a heading, a short paragraph, and a mapped list of hobbies

Make sure your Vite dev server is running before you start — `npm run dev`.

## Step 1 — Open the File

Open `src/exercises/lesson-01/studentWork.jsx`.

You'll see a starter component that looks like this:

```
export default function StudentWork() {  
  //add variables here  
  return (  
    <div>  
      {/* add JSX here */}  
      <p>Student output will go here</p>  
    </div>  
  );  
}
```

You'll replace the placeholder content in the next two steps.



## Step 1 (continued) — Add Variables

Inside the component, above the `return`, add your variables:

```
export default function StudentWork() {  
  const name = "Alex";  
  const age = 28;  
  const hobbies = ["reading", "hiking", "cooking"];  
  
  return (  
    <div>  
      {/* add JSX here */}  
    </div>  
  );  
}
```

Save and confirm no errors appear in the browser or terminal.

## Step 2 — Return JSX

Now fill in the `return` with a heading, paragraph, and list:

```
return (  
  <div>  
    <h1>About Me</h1>  
    <p>Hi! My name is {name} and I am {age} years old.</p>  
    <ul>  
      {hobbies.map((hobby, index) => (  
        <li key={index}>{hobby}</li>  
      ))}  
    </ul>  
  </div>  
);
```

Save — your About Me page should now display in the browser.

## Extension — Stretch Your Component

If you finish early, try adding more to your About Me page:

- A **favorite quote** as a `const` string, rendered in a `<blockquote>`
- A **skills list** — a second array mapped to a list
- A **link** to your GitHub profile using an `<a>` tag

There's no single right answer — experiment and see what renders!

# Wrap-Up

This week you learned:

- React is a declarative library for building UIs out of components
- The Virtual DOM makes updates efficient
- Vite scaffolds and serves React projects locally
- JSX lets you write HTML-like syntax inside JavaScript functions

**Before next session:** Complete the weekly assignment — set up your GitHub repo, scaffold a Vite project, and render a todo list. Submit your PR link in CTD Learns. Post questions in **#[class-name]-discussion** on Slack.